CloudLens AI

Project summary: what we built, and how it works

A plain-language walkthrough of the CloudLens AI prototype: the idea, the data, the four parts that do the thinking, the website, and every step taken so far.

Project	Status	Date
B.Tech I.T. final-year major project	**Working prototype, live online**	**14 October 2026**

At a glance

CloudLens in one minute

Companies rent servers, databases and storage from cloud providers such as Amazon (AWS), Microsoft (Azure) and Google. They pay every month, but a large share of that money often goes to machines nobody is using. CloudLens reads the bill, finds that waste, explains it in everyday words, predicts next month's bill, and lets you approve fixes.

46
cloud resources in the sample company, across 3 providers

$10,631
sample monthly cloud bill

$2,961
a month found as waste (28% of the bill)

0.82%
average daily error of the bill prediction

What a user can do

- **Sign up and log in** to their own private account.
- **Connect cloud accounts** (AWS, Azure, Google Cloud). A new account is empty until it connects one.
- **See where the money goes**: total bill, spend per team, daily spending over the year.
- **See ways to save**: every wasteful resource, why it was flagged, and how much fixing it saves.
- **Approve or skip each fix**, and see the savings add up in a history log.
- **See the next three months' bills** before they arrive.
- **Ask questions in plain English**, such as "Which servers are not being used?"

> **Honest scope.** As agreed in the project guide, this prototype runs on realistic *sample* data rather than a real company's cloud. Nothing real is ever switched off, and all savings are estimates. This keeps the project free and safe, and real cloud integration is listed as the next step.

Contents

1 The problem and the idea

2 How CloudLens works, step by step

3 The sample data

4 Finding waste

5 Predicting future bills

6 The question-answering assistant

7 The website

8 Accounts, cloud connections and the demo account

9 How the system is built and hosted

10 Testing and quality checks

11 The journey so far

12 Limits and next steps

1 · The problem and the idea

Why cloud bills need a lens

Renting computers from a cloud provider is like paying an electricity bill for a building where nobody turns the lights off. Teams start servers for a project and forget them. They pick a bigger machine “just in case”. Test machines that are only needed in office hours run all night and every weekend. Old storage disks sit unattached, still billed every month.

Cloud bills are long and technical, so this waste is hard to spot. CloudLens is a tool that looks through the bill for you and answers three simple questions:

Where is it wasted?

Find the waste

Flags every resource that costs money without doing useful work, with the reason and the evidence.

What will it cost?

Predict the bill

Uses machine learning to forecast the next three months, and how much lower they would be after the fixes.

What should I do?

Explain and act

Explains each saving in plain words, answers questions, and lets the user approve fixes safely.

The four kinds of waste CloudLens looks for

Kind of waste	Everyday explanation	Suggested fix
Switched on but not used	A server that is running and billed, but doing almost no work.	Switch it off
Bigger than it needs to be	Even on its busiest day it uses less than half of the power it pays for, like renting a lorry to carry one box.	Move to the next smaller size (half the price)
Left on when nobody is working	A test or development machine used in office hours but left running every night and weekend.	Switch off outside 8am–8pm on weekdays
Storage nobody uses	A disk connected to nothing, which nobody has read or written for a month.	Back it up, then delete it

2 · How CloudLens works

From a bill to a decision, step by step

Every number on the screen comes out of this pipeline. Each box is a separate, small part of the program, which makes it easy to test and to explain.

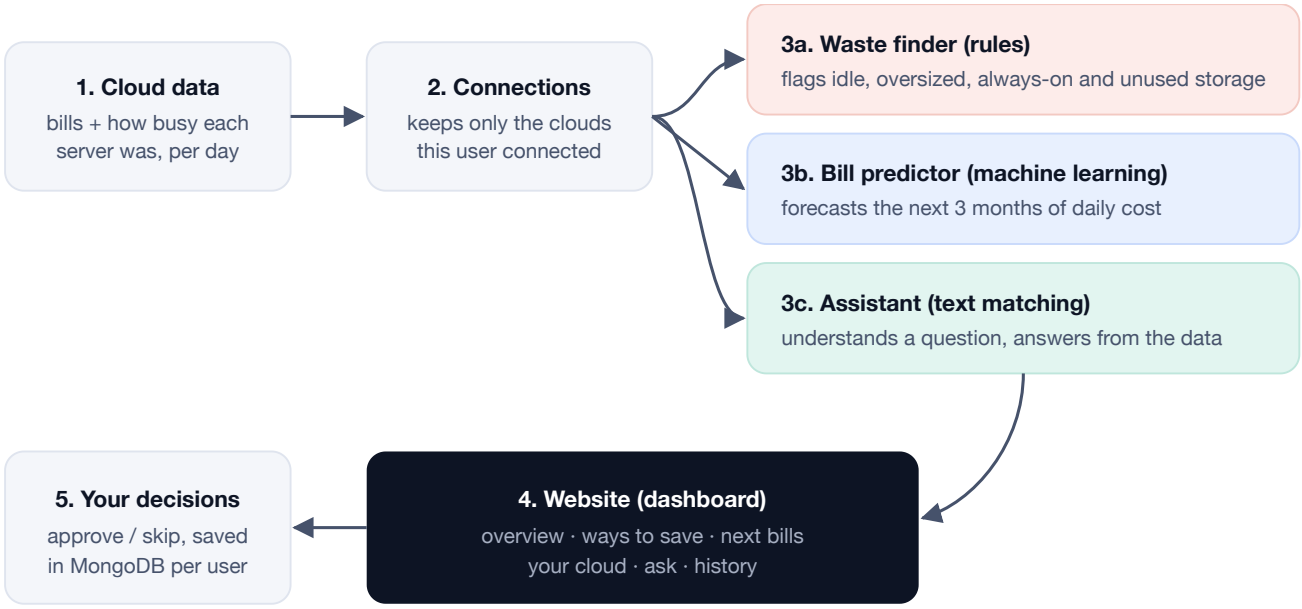


Figure 1. The CloudLens pipeline. The three engines in the middle all read the same data, and the website shows their results.

1. **Cloud data.** A year of daily records: what each resource cost and how busy its processor and memory were.
2. **Connections.** Each user only sees data from the cloud accounts they have connected.
3. **Three engines** turn the raw data into answers: a rule-based waste finder, a machine-learning bill predictor, and a question-answering assistant.
4. **Website.** Shows the results in simple screens with charts.
5. **Decisions.** When the user approves or skips a fix, it is saved in the database and the savings totals update everywhere.

3 · The sample data

A realistic company, built from scratch

Using a real company's cloud would cost money and need private access, so we wrote a small Python program, `generate_data.py`, that creates a realistic sample company. It always produces exactly the same data (a fixed "random seed"), so every result in this document can be reproduced.

46
resources: servers, databases, storage disks

365
days of history, Oct 2025 to Sep 2026

15,520
daily records of cost and usage

$118,671
total spend over the year

Who owns what

Cloud provider	Teams	Resources
Amazon Web Services (AWS)	Platform, Payments	19
Google Cloud	Search, Analytics	18
Microsoft Azure	Mobile	9

The clever part: waste was planted on purpose

While generating the company, we deliberately made **20 of the 46 resources wasteful** and wrote down which ones in a separate "answer key" file. The waste finder never reads this file. Afterwards, we compare its findings with the answer key, which lets us *measure* how accurate it is instead of just claiming it works.

Planted waste	How many
Switched on but not used	5
Bigger than it needs to be	6
Left on when nobody is working	5
Storage nobody uses	4

> **Trick cases.** We also added healthy resources that *look* wasteful at first glance, for example a cache server with a quiet processor but full memory, or a nightly batch server that is quiet by day and very busy at night. A careless rule would wrongly flag them; a good one must not.

The data also has realistic patterns: costs grow as new servers are added during the year, traffic is lower at weekends, and three one-day cost spikes were added.

4 · Finding waste

Simple rules that anyone can check

The waste finder looks at the last 30 days of each resource and applies four clear rules, in order. We chose rules over a “black box” model on purpose: every finding can be explained in one sentence, and every threshold sits in one place in the code.

Rule	When it triggers	How the saving is worked out
Not used	Average processor use under 5%, never above 15%, and memory under 20%	The whole monthly cost
Left on at night	A test/development machine, on 20+ hours a day, busy in office hours but under 5% at night	The cost of the hours outside 8am–8pm on weekdays (about 64%)
Too big	Busiest processor moment under 40% and busiest memory under 50%	Half the cost (the next size down costs half)
Unused storage	Connected to nothing and no reads or writes for 30 days	The whole monthly cost

How accurate is it?

20 / 20
planted problems found

0
wrong alarms on healthy resources, including the trick cases

$2,961
a month of estimated savings found

In technical terms, precision and recall are both 100%. Precision means “when it flags something, is it right?”; recall means “of all the real problems, how many did it catch?”.

> **Why we don’t oversell this.** The same team wrote both the data and the rules, so a perfect score shows the rules work *as designed*. It does not prove they would score 100% on a real company’s bill. We say this openly in the app and in this document.

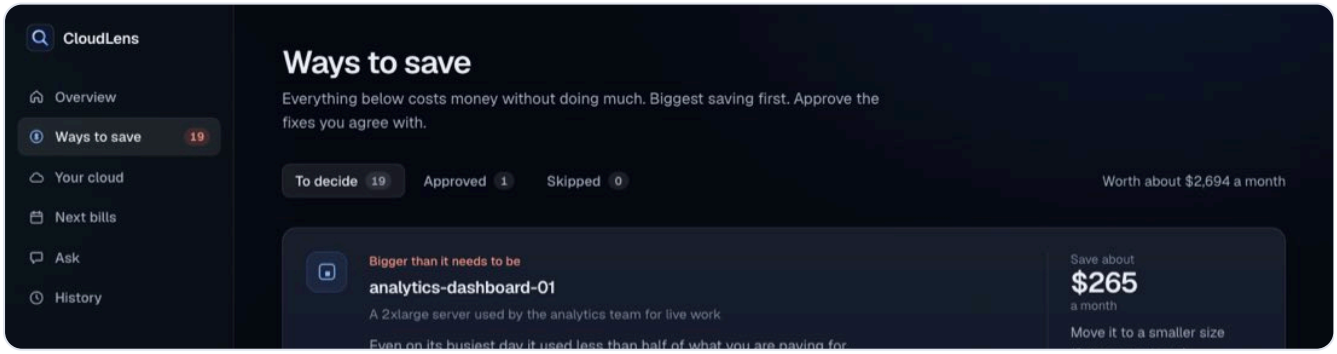


Figure 2. The “Ways to save” screen: each finding shows the reason, the evidence (“How we know”), the saving, and Approve / Skip buttons.

5 · Predicting future bills

Machine learning, tested fairly

The bill predictor uses **linear regression** from the scikit-learn library. In simple terms, it learns a formula for the daily bill from two things:

- **How much is switched on that day** (the list price of all resources that exist), and
- **Which day of the week it is**, because weekends are cheaper.

For the future, it assumes the set of resources stays as it is today, and predicts the next 92 days (to the end of December 2026), with a shaded “likely range” around the prediction.

The fair test

We hid three separate two-month periods from the model, asked it to predict them, and compared the predictions with what really happened. We also compared it against two simple methods that use no machine learning at all, to check the model is actually worth having.

Method	Average daily error	In dollars per day
CloudLens model (machine learning)	**0.82%**	**$2.94**
Simple guess: copy last week	1.12%	$3.96
Simple guess: average of last week	1.63%	$5.79

The model’s error is about **27% lower** than the best simple method.

> **A useful lesson from the build.** Our first model used “time” as its main input and actually *lost* to the simple “copy last week” method. Replacing time with “how much is switched on” fixed it. Testing against simple baselines is what revealed the problem, which is a good point to make in the viva.

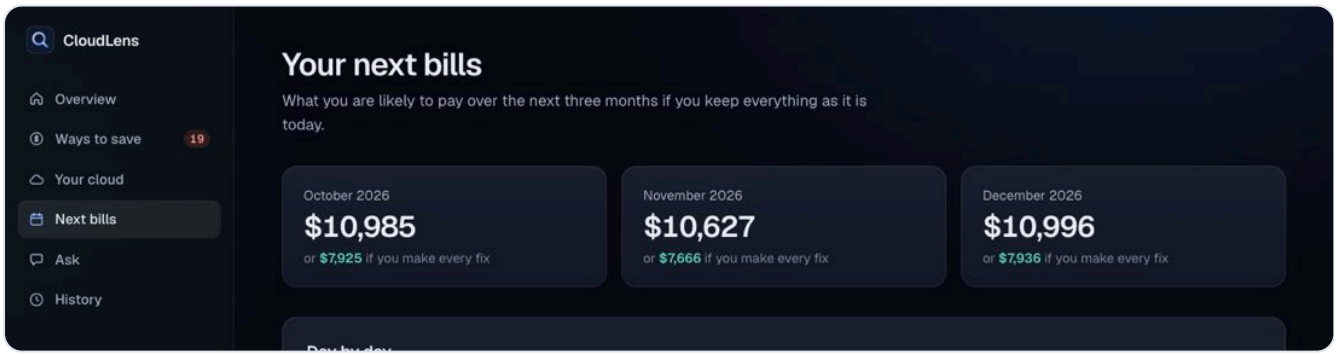


Figure 3. The “Next bills” screen: the next three months, what they would be after every fix, and the day-by-day chart with the likely range.

6 · The question-answering assistant

Plain-English answers, with no paid AI

The project guide asked us to avoid paid AI services in the first version. So the assistant works in two simple steps:

1. **Understand the question.** It knows 12 topics (for example “how much can we save”, “which servers are not used”, “what will we spend next month”), each with a few example phrasings. A technique called **TF-IDF with cosine similarity** (from scikit-learn) measures how similar the user’s words are to each example and picks the closest topic. This is why “any servers we could shut down?” still matches “Which servers are not being used?”.
2. **Answer from live data.** The answer is filled in from the current numbers, so it is always correct for that user’s connected clouds.

If a question is unrelated (for example about cricket), it says so politely and suggests questions it can answer. Under each answer it shows which topic it matched and how closely, so the reasoning is visible.

The 12 topics

- How much did we spend?
- How much can we save?
- Which resources cost the most?
- Which team spends the most?
- Which servers are not being used?
- Which are bigger than they need to be?
- Which machines are left on all night?
- Is there storage nobody is using?
- What will we spend next month?
- How are savings calculated?
- Which fixes have I approved?
- Is this real data?

7 · The website

A complete product, not just a dashboard

The website works like a real software product: a public homepage, sign-up and login, and a private area with seven screens. The design is dark, clean and calm, and every label uses everyday words (“Switched on but not used” rather than “idle instance”). Technical details such as precision, recall and error rates are kept in folded “Technical details” sections for those who want them.

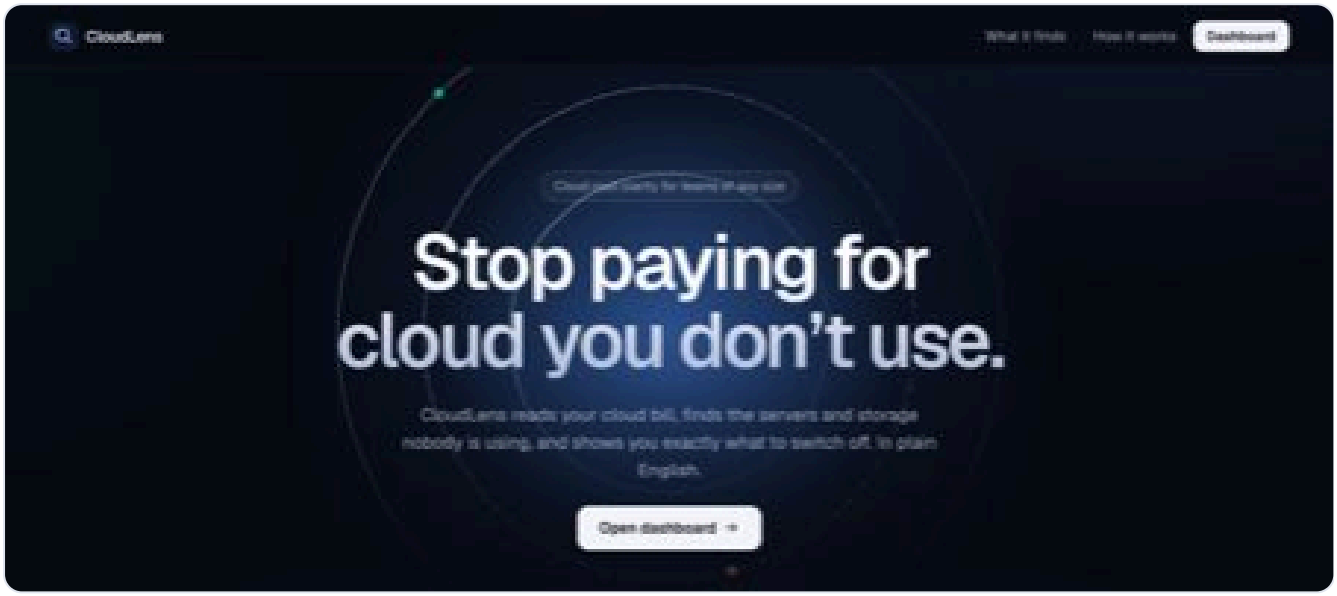


Figure 4. The public homepage, with a live preview of the sample company’s waste.

Screen	What it shows
Overview	Money that could be saved, the bill as a bar (useful, wasted, already fixed), next month’s bill, waste by kind, daily spend chart, spend per team.
Ways to save	Every finding with its reason and evidence; Approve or Skip; tabs for decided items.
Your cloud	All resources with cloud, team, purpose, processor and memory use, cost and any problem; search and filters.
Next bills	Three-month forecast, with and without the fixes, plus how the prediction was tested.
Ask	The question-answering assistant with suggested questions.
History	Every decision made, with monthly and yearly savings; any decision can be undone.
Connections	Connect or disconnect AWS, Azure and Google Cloud.

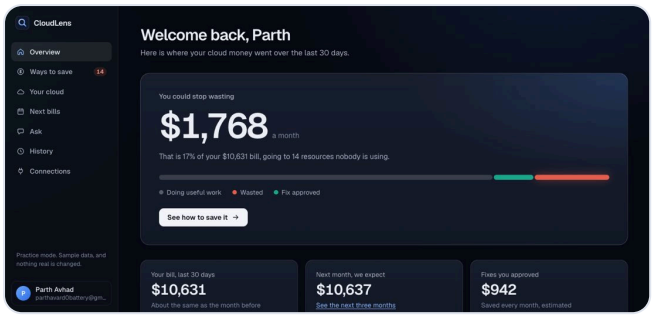


Figure 5. Overview screen for the showcase account: $1,768 a month still to fix and $942 a month already approved (green on the bar).

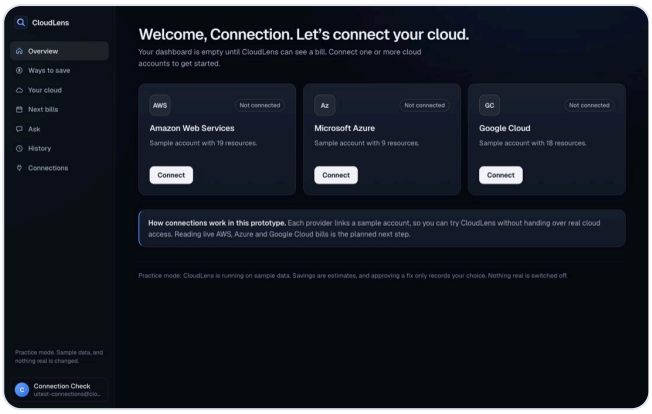


Figure 6. A brand-new account starts empty and is asked to connect a cloud first.

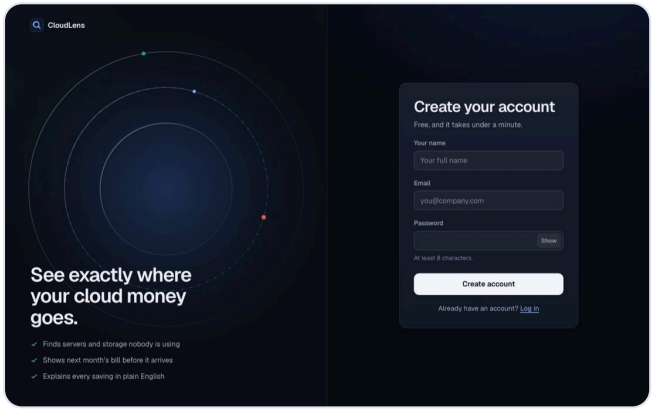


Figure 7. Sign-up screen.

Design choices, briefly

- **Colour has meaning:** blue is spending, coral is waste, green is waste already dealt with.
- **Works on phones:** the menu moves to a bar at the bottom of the screen.
- **Accessible:** keyboard focus is visible, text has strong contrast, and chart colours were checked for colour-blind readers.
- **Light and fast:** charts are drawn with simple code, not a heavy chart library.
- **Safe by design:** every “Approve” asks for confirmation and says clearly that nothing real will change.

Every user has their own private space

Accounts and security

- Accounts are stored in **MongoDB Atlas**, a cloud database (free tier).
- **Passwords are never stored.** Only a scrambled “hash” is kept (bcrypt), so even the database owner cannot read them.
- After login, the browser keeps a signed pass (a JWT) in a cookie that page scripts cannot read, which protects against common attacks.
- Each user’s connections and decisions are separate; one user can never see or undo another’s.

Cloud connections

A new account starts empty. On the Connections screen the user picks AWS, Azure or Google Cloud. A short four-step progress view (linking, reading 12 months of bills, checking usage, looking for waste) then opens a dashboard that shows **only** the connected clouds. For example, AWS alone shows a $3,552 bill with $977 of waste.

> **What “connect” means today.** Each provider links its part of the sample company. The app never asks for real cloud passwords or keys, and the connect window says so. Connecting a real AWS, Azure or Google account through their billing systems is the planned next step.

The showcase account for faculty

One account, **parthavhad04@gmail.com**, is prepared for demonstrations (the password is shared separately). It already has all three clouds connected and a short history: five approved fixes worth about $942 a month and one skipped fix, with 14 findings left so a fix can be approved live during the demo. A small script, `seed_demo.py`, resets it to this state in seconds before each presentation.

9 · How the system is built and hosted

The technical picture

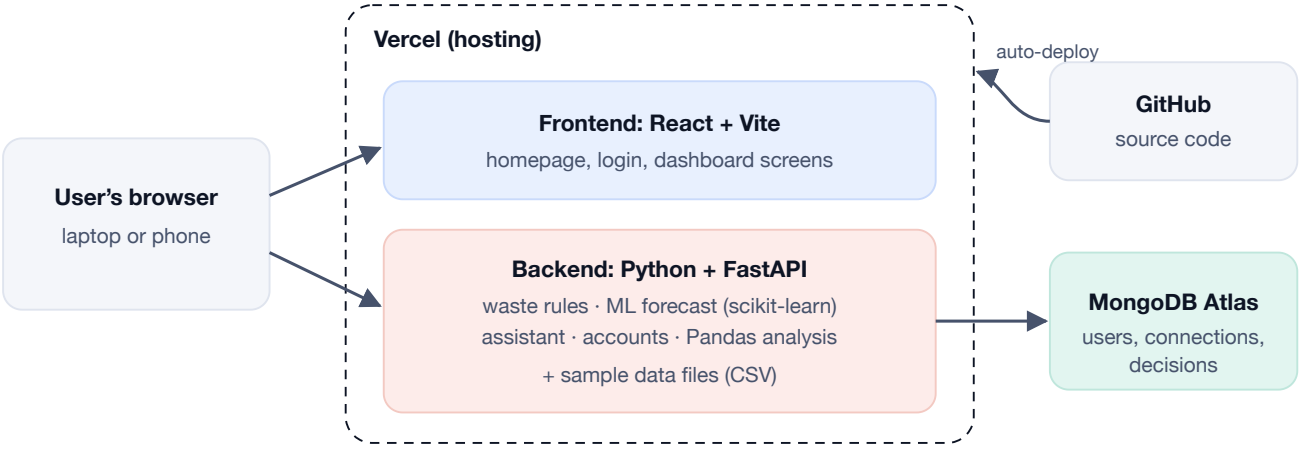


Figure 8. Where each part runs. Pushing code to GitHub automatically redeploys the site on Vercel.

Part	Tool	Why we chose it
Website	React + Vite	Fast, popular, matches the project guide
Server logic	Python + FastAPI	Simple to write; automatic API documentation page
Data analysis	Pandas	Standard tool for tables of data
Machine learning	scikit-learn	Free, well documented, explainable models
Database	MongoDB Atlas (free tier)	Stores accounts and decisions online
Security	bcrypt, JWT cookies	Industry-standard password and login handling
Code and hosting	GitHub + Vercel	Free; every change goes live automatically
Testing	pytest, Playwright	Automatic checks of logic and of the real website

Live site: cloudlens-sooty.vercel.app · **Code:** github.com/AvhadParth/CloudLens (private)

How we know it works

47
automatic tests, all passing

2
screen sizes checked in a real browser (desktop and phone)

0
secrets stored in the code repository

What the automatic tests check

Area	Examples of what is tested
Numbers	Totals on screen add up to the raw data; every breakdown adds up to the same bill.
Waste finder	Finds the planted waste; never flags the trick cases; a saving is never bigger than the cost.
Forecast	Never trains on the period it is tested on; starts the day after the data ends; rejects silly requests.
Assistant	Every suggested question reaches its own topic; reworded questions still match; unrelated questions get a polite fallback.
Accounts	Private pages need a login; passwords are hidden; duplicate emails and weak passwords are refused; a tampered login pass is rejected.
Connections	A new account sees nothing; connecting one cloud shows only that cloud; users never see each other’s data.
Decisions	Approve then undo works; deciding twice keeps one record; healthy resources cannot be “fixed”.

Tests use a temporary in-memory database, so running them never touches the real accounts. After each change to the website we also opened it in an automated browser (Playwright), took screenshots at desktop and phone width, clicked through sign-up, connecting a cloud and approving a fix, and checked the browser for errors.

11 · The journey so far

What we did, in order

1. **Planned the project**
   Read the project guide and turned its seven steps into a full plan with a clear, honest scope.

2. **Created the sample company**
   Wrote the data generator: 46 resources, a year of daily data, 20 planted problems and an answer key.

3. **Built the backend and the waste finder**
   FastAPI server, four explainable rules, savings estimates, and accuracy scoring against the answer key.

4. **Added machine learning**
   Built the bill predictor, tested it against simple baselines, found it lost at first, and fixed it so it wins.

5. **Added the assistant and safe “approve” actions**
   Text-matching assistant with 12 topics; approve, skip and undo, all recorded and never touching real servers.

6. **Designed the website, three times**
   Version 1 was too plain; version 2 was too playful; version 3 is the clean, premium design used now, with everyday wording.

7. **Made it a complete product**
   Homepage, sign-up, login and private accounts stored securely in MongoDB Atlas.

8. **Put it online**
   Code on GitHub; website on Vercel, deploying automatically on every change; fixed the database network setting so the live site can log people in.

9. **Added cloud connections**
   New accounts start empty and connect AWS, Azure or Google Cloud; every screen shows only the connected clouds.

10. **Prepared the faculty demo**
    A showcase account with all clouds connected and a decision history, plus a one-line reset script.

Being clear about what it is, and where it can go

Current limits

- **Sample data only.** The company, its bills and its waste are generated. Results on real bills would differ.
- **Simulated actions.** Approving a fix records the decision; it does not change any real server.
- **Sample connections.** “Connect AWS” links the sample AWS account, not a real one.
- **Simple assistant.** It answers 12 kinds of question well, but cannot hold an open-ended conversation.
- **Account basics only.** No “forgot password”, email verification or limit on login attempts yet.

Possible next steps

Real cloud connections

Read real bills through the AWS Cost Explorer, Azure Cost Management and Google Cloud Billing interfaces, using read-only access.

Spotting sudden cost spikes

Flag unusual days automatically (the sample data already contains three planted spikes to test against).

“What if” controls

Let users change the thresholds and see how the findings and savings change.

Account features

Password reset, email verification and protection against repeated login attempts.

Still to come for submission: the project blackbook (formal report). This summary, the diagrams, the test results and the screenshots in it can be reused there.